

Exercises - Classes:

1. Use the **Coin** class provided in the file **coin_module.py**. You will edit the tester program called **coinprog.py** so that it is able to produce input / output as follows:

```
Type:
    f - to flip the coin
    q - to quit

Enter selection: f
The result was H

Type:
    f - to flip the coin
    q - to quit

Enter selection: f
The result was T

Type:
    t - to flip the coin
    q - to quit

Enter selection: q
```

2. Write an adaption of the **Coin** class called **Dice**. “**Dice**” should represent a dice that can be thrown (with six faces) rather than a coin that can be flipped. There should be:
 - a **cast()** method (in place of **Coin**’s **flip()** method)
 - a **get_face()** method which should return an integer representing the value of the dice when it is thrown.
 - a static constant which is a dictionary that can be used to convert the face value to a word.

The new class should be compatible with a test program **diceprog.py**, (which you should also write) so that **diceprog.py** is able to produce input/output such as:

```
Type:
    t - to cast the dice
    q - to quit

Enter selection: t
The result was five

Type:
    t - to cast the dice
    q - to quit

Enter selection: q
```

3. Use the **Coin** class provided. You will write a tester program called **coinlistprog.py** which fills a list with 20 **Coin** objects and prints the percentages of heads and tails. **coinlistprog.py** should produce output as follows:

```
Coins: T T H T H H T T T T T H H T T T T H H H  
Heads: 40.00%  
Tails: 60.00%
```

4. Use the class **Employee** in the file **employee_module.py** and the **employeeprog.py** program. Modify the **Employee** class by adding a property getter and setter for a new attribute: **jobTitle** which contains the job title for the employee. Use the setter (mutator) to enforce the invariant that **jobTitle** must be at least 5 characters long and may not contain digits. Test the **Employee** class by compiling and running the **employeeprog.py** program.
5. Use the class **Employee** and the **employeeprog.py** program from question 1. Extend the **Employee** class by adding new methods **hire()**, **fire()**, and **isHired()** so that it is possible to hire and fire the employee. At times when the employee is fired, his/her salary should be fixed at zero (i.e. it should not be possible to promote, demote or raise the salary of someone who is unemployed). **isHired()** should indicate whether an employee is currently hired or not, you will need to add an extra attribute, **isHired**, to the class. You must use a property getter and setter to manage **isHired**. Extend the **employeeprog.py** program appropriately to test the new methods.

These problems are adapted from a course at UCL in the Department of Information Studies, [Programming 1](#).

Further practice problems can be found in the textbook [Think Like a Computer Scientist](#) or the interactive version [How to Think Like a Computer Scientist: Interactive Edition](#)